Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «**Криптография и сетевая безопасность**»
Тема: **«Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»**

Выполнил:
Студент 4 курса
Группы АС-65
Касьяник К. А.
Проверил:
Хацкевич А. С.

Брест 2026

Цель: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 7

1. Генерация и тестирование псевдослучайных последовательностей (файл "Лабораторная работа №1 (ч.1)")

В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

Требования:

- реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- подобрать параметры генератора (*a*, *c*, *N*, *x_0* и др.) так, чтобы добиться максимального периода последовательности *T_max* (согласно теоретическим условиям из методички);
- провести базовое статистическое тестирование полученной последовательности (например, оценить частотность появления элементов, равномерность распределения).

2. Тестирование на простоту и генерация больших простых чисел (файл "Лабораторная работа №1 (ч.2)")

На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту.

Требования:

- реализовать стандартную схему генерации кандидата в простые числа:
    - сгенерировать случайное *p*-битовое число *n* (используя генератор из Этапа 1);
    - установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
    - выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (2, 3, 5, 7, ..., вплоть до 256 или 2000), что отсекает до 80% составных чисел без тяжелых тестов;
- интегрировать целевой тест из таблицы вариантов (вместо стандартного Рабина-Миллера или в дополнение к нему) для финальной проверки кандидата. (Примечание: для метода Михалеску используется собственная специфическая схема генерации доказуемо простых чисел).

main.py:

```
import time
from math import ceil, isqrt

# Часть 1 LFSR
class LFSR:
    def __init__(self, poly, state):
        self.k = max(poly)
        self.taps = [e for e in poly if e < self.k]
        self.state = state

    def bit(self):
        out = self.state & 1
        new = 0
        for e in self.taps:
            new ^= (self.state >> e) & 1
        self.state = (self.state >> 1) | (new << (self.k - 1))
        return out
    def bits(self, n):
        return [self.bit() for _ in range(n)]
    def num(self, n):
        v = 0
        for _ in range(n):
            v = (v << 1) | self.bit()
        return v

def period(poly, init):
    g = LFSR(poly, init)
    t = 0
    while True:
        g.bit()
        t += 1
        if g.state == init:
            return t

def chi2(bits, m):
    blocks = len(bits) // m
    cnt = [0] * (1 << m)
    for i in range(blocks):
        w = 0
        for b in bits[i * m:(i + 1) * m]:
            w = (w << 1) | b
        cnt[w] += 1
    exp = blocks / (1 << m)
    return sum((c - exp) ** 2 / exp for c in cnt)
POLYS = [
    ("x^6+x^3+1", [6, 3, 0]),
    ("x^6+x^2+x+1", [6, 2, 1, 0]),
    ("x^6+x+1", [6, 1, 0]),
]
CRIT = {1: 3.841, 2: 7.815, 3: 14.067}

def part1():
    print("Многочлен | Периоды по всем ненулевым начальным состояниям | При-
митивный")
    for name, p in POLYS:
        k = max(p)
        periods = sorted({period(p, s) for s in range(1, 1 << k)})
        print(f"{name:14} {periods} {'да' if max(periods) == (1 << k) - 1 else
'нет'}")
    print("\nX2 (начальное состояние 1); + равномерность не отвергается, -
отвергается")
    print("крит. значения (alpha=0.05): m=1: 3.84, m=2: 7.81, m=3: 14.07")
```

```
    Ns = (1000, 10000, 100000)
    print(f"{'многочлен':14} m " + " ".join(f"N={n:<9}" for n in Ns))
    for name, p in POLYS:
        for m in (1, 2, 3):
            row = []
            for n in Ns:
                x2 = chi2(LFSR(p, 1).bits(n), m)
                row.append(f"{x2:8.1f}{'+' if x2 < CRIT[m] else '-'}  ")
            print(f"{name:14} {m} " + " ".join(row))

# Часть 2 метод Михалеску
PRIME_POLY = [19, 5, 2, 1, 0]
E = 1 / 3
B = 32
S = 2

def small_primes(limit):
    sieve = [True] * limit
    res = []
    for i in range(2, limit):
        if sieve[i]:
            res.append(i)
            for j in range(i * i, limit, i):
                sieve[j] = False
    return res

def rabin_miller(n, bases):
    if n < 4:
        return n in (2, 3)
    if n % 2 == 0:
        return False
    s, t = 0, n - 1
    while t % 2 == 0:
        t //= 2
        s += 1
    for a in bases:
        if a % n == 0:
            continue
        x = pow(a, t, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True

def trial_prime(m, g):
    while True:
        n = g.num(m) | (1 << (m - 1)) | 1
        if all(n % d for d in range(2, isqrt(n) + 1)):
            return n

def mihailescu(m, g, st, sp):
    if m < B:
        return trial_prime(m, g), None

    F, _ = mihailescu(ceil(E * m) + 1, g, st, sp)

    lo = (1 << (m - 2)) // F + 1
    hi = (1 << (m - 1)) // F - S * m
```

```
    while True:
        t = lo + g.num(m) % (hi - lo)
        n0 = 2 * F * t + 1

        for i in range(S * m + 1):
            n = n0 + 2 * F * i
            st["cand"] += 1
            if any(n % q == 0 for q in sp):
                st["sieve"] += 1
                continue
            bases = [2 + g.num(16) % min(n - 3, 1000) for _ in range(5)]
            if rabin_miller(n, bases):
                return n, F
            st["rm"] += 1


def gen_prime(m, limit, g):
    st = {"cand": 0, "sieve": 0, "rm": 0}
    n, F = mihailescu(m, g, st, small_primes(limit))
    return n, F, st

def part2(bits, limit):
    g = LFSR(PRIME_POLY, time.time_ns() % ((1 << 19) - 1) + 1)
    t0 = time.perf_counter()
    n, F, st = gen_prime(bits, limit, g)
    dt = time.perf_counter() - t0
    print(n)
    print(f"F = {F}")
    print(f"бит: {bits}, кандидатов: {st['cand']}, отсеяно просеиванием (<{limit}):
{st['sieve']}, "
          f"отсеяно Рабином-Миллером: {st['rm']}, время: {dt:.4f} с")

# Графики
def plots():
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    Ns = [100, 300, 1000, 3000, 10000, 30000, 100000]
    fig, axs = plt.subplots(1, 3, figsize=(15, 4))
    for ax, m in zip(axs, (1, 2, 3)):
        for name, p in POLYS:
            bits = LFSR(p, 1).bits(Ns[-1])
            ax.plot(Ns, [chi2(bits[:n], m) for n in Ns], marker="o", label=name)
        ax.axhline(CRIT[m], color="k", linestyle="--", label="критическое (0.05)")
        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.set_title(f"блоки по {m} бит")
        ax.set_xlabel("длина последовательности N")
        ax.set_ylabel("X2")
        ax.legend()
    fig.tight_layout()
    fig.savefig("lab1_st1.png", dpi=120)
    bits_list = [64, 128, 256, 512]
    runs = 100
    times, sv, rm = [], [], []
    for b in bits_list:
        t = s = r = 0
        for i in range(runs):
            g = LFSR(PRIME_POLY, 1000 * (i + 1) + b)
            t0 = time.perf_counter()
            _, _, st = gen_prime(b, 2000, g)
            t += time.perf_counter() - t0
            s += st["sieve"]
```

```
            r += st["rm"]
        times.append(t / runs)
        sv.append(s / runs)
        rm.append(r / runs)
    fig, axs = plt.subplots(1, 2, figsize=(11, 4))
    axs[0].plot(bits_list, times, marker="o")
    axs[0].set_ylabel("среднее время, с")
    axs[1].plot(bits_list, sv, marker="o", label="отсеяно просеиванием (<2000)")
    axs[1].plot(bits_list, rm, marker="o", label="отсеяно Рабином-Миллером")
    axs[1].set_ylabel("кандидатов, среднее")
    axs[1].legend()
    for ax in axs:
        ax.set_xlabel("разрядность p")
    fig.tight_layout()
    fig.savefig("lab1_st2.png", dpi=120)

# Тесты корректности
REF_BASES = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
def check(name, cond):
    print(f"{name}: {'OK' if cond else 'FAIL'}")
def tests():
    check("период x^19+x^5+x^2+x+1 = 2^19-1", period(PRIME_POLY, 1) == (1 << 19) -
1)
    sp = set(small_primes(10000))
    bad = [n for n in range(2, 10000) if rabin_miller(n, [2, 3, 5, 7, 11]) != (n in
sp)]
    check("Рабин-Миллер совпадает с решетом для n < 10000", not bad)

    carm = [561, 1105, 1729, 2465, 2821, 6601, 8911]
    check("Числа Кармайкла определяются как составные", not any(rabin_miller(n, [2,
3, 5, 7, 11]) for n in carm))

    g = LFSR(PRIME_POLY, 12345)
    for m in (32, 64, 128, 256):
        n, F, st = gen_prime(m, 2000, g)
        ok = (n.bit_length() == m
              and (n - 1) % F == 0
              and F > 2 ** (E * m)
              and rabin_miller(n, REF_BASES)
              and rabin_miller(F, REF_BASES))
        check(f"Михалеску {m} бит: разрядность, F | n-1, F > 2^(em), n и F
простые", ok)
    n = gen_prime(32, 2000, g)[0]
    check("32-битное число простое (проверка делением)", all(n % d for d in
range(2, isqrt(n) + 1)))

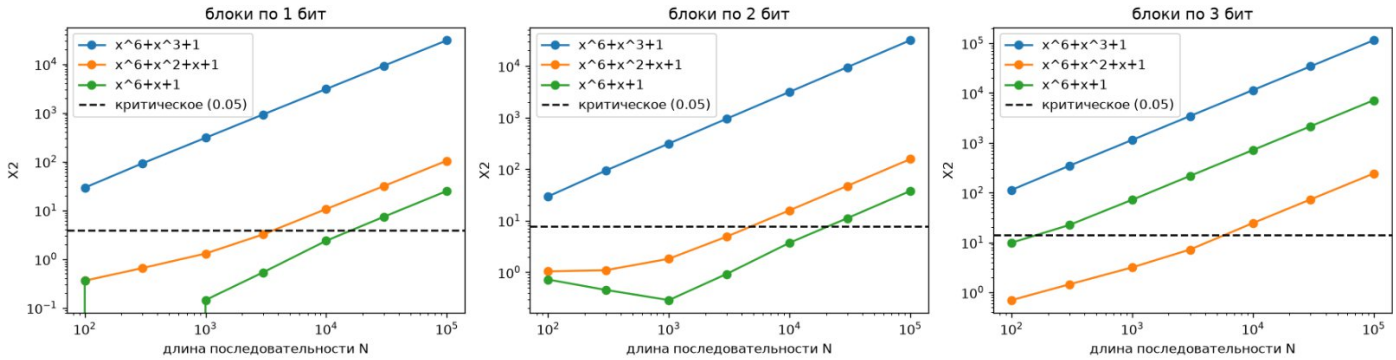
if __name__ == "__main__":
    while True:
        cmd = input("\n1 - часть1, 2 - часть2, 3 - тесты, 4 - графики, 0 - выход:
").strip()
        if cmd == "1":
            part1()
        elif cmd == "2":
            bits = input("Разрядность (Enter = 256): ").strip()
            limit = input("Просеивание до (Enter = 2000): ").strip()
            part2(int(bits) if bits else 256, int(limit) if limit else 2000)
        elif cmd == "3":
            tests()
        elif cmd == "4":
            plots()
        elif cmd == "0":
            break
```

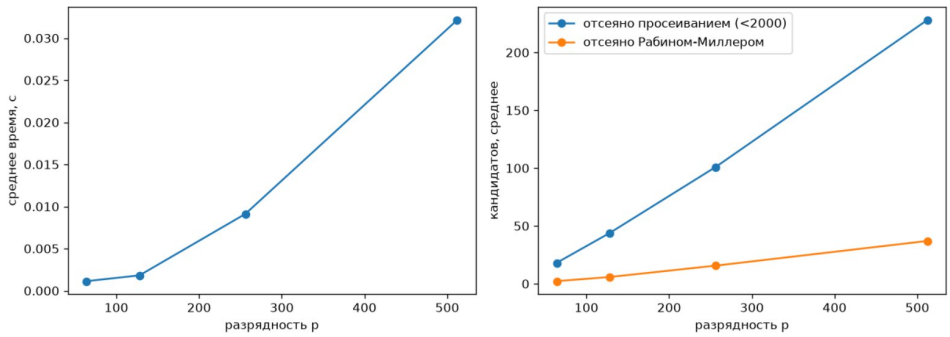
```
Многочлен | Периоды по всем ненулевым начальным состояниям | Примитивный
x^6+x^3+1       [9] нет
x^6+x^2+x+1     [1, 31] нет
x^6+x+1         [63] да

X2 (начальное состояние 1); + равномерность не отвергается, - отвергается
крит. значения (alpha=0.05): m=1: 3.84, m=2: 7.81, m=3: 14.07
многочлен       m N=1000      N=10000     N=100000
x^6+x^3+1       1    306.9-     3084.7-    30862.5-
x^6+x^3+1       2    312.8-     3146.1-    31479.4-
x^6+x^3+1       3   1147.0-    11480.3-   114813.7-
x^6+x^2+x+1     1      1.3+       10.6-      104.1-
x^6+x^2+x+1     2      1.8+       15.9-      156.2-
x^6+x^2+x+1     3      3.2+       24.6-      242.7-
x^6+x+1         1      0.1+        2.4+       25.0-
x^6+x+1         2      0.3+        3.7+       37.6-
x^6+x+1         3     72.2-      718.9-     7182.0-
```

```
Разрядность (Enter = 256):
Просеивание до (Enter = 2000):
896510530133278573713228852910250565962441085551226522364410306219189893892 3
F = 1419235125277657489118324 33
бит: 256, кандидатов: 6, отсеяно просеиванием (<2000): 3, отсеяно Рабином-Миллером: 1, время: 0.0046 с
```

```
период x^19+x^5+x^2+x+1 = 2^19-1: OK
Рабин-Миллер совпадает с решетом для n < 10000: OK
Числа Кармайкла определяются как составные: OK
Михалеску 32 бит: разрядность, F | n-1, F > 2^(em), n и F простые: OK
Михалеску 64 бит: разрядность, F | n-1, F > 2^(em), n и F простые: OK
Михалеску 128 бит: разрядность, F | n-1, F > 2^(em), n и F простые: OK
Михалеску 256 бит: разрядность, F | n-1, F > 2^(em), n и F простые: OK
32-битное число простое (проверка делением): OK
```





Вывод: освоили программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.